

Amazon Bedrock Mantle invocation audit logs in CloudWatch Logs

Contents

1. Choose the logging mechanism	2
What “from the client side” means	2
2. Prerequisites and scope	2
3. What Mantle records	4
4. Option A — Configure with the AWS console	4
Step 1 — Create or select a trail	4
Step 2 — Enable CloudWatch Logs delivery	5
Step 3 — Select Mantle inference data events	5
5. Option B — Configure from a workstation with AWS CLI	5
5.1 — Set the account, Region, and resource names	6
5.2 — Create the dedicated S3 bucket	6
5.3 — Create the CloudWatch log group	7
5.4 — Create the CloudTrail delivery role	7
5.5 — Create the advanced event selector file	8
5.6 — Create the trail, apply selectors, and start logging	9
5.7 — Verify the configuration	9
6. If you already have a trail	10
7. Send an invocation from your client	10
Existing application	11
Python smoke test using Gemma 4 31B	11
8. Find the event in CloudWatch Logs	12
Console and Logs Insights	12
CLI	13
Expected event shape	13
Completion criteria	13
9. Troubleshooting	14
10. Operational metrics, retention, and cost	14
CloudWatch metrics are separate	14
Charges and retention	15
11. If you require client-side prompt and response logs	15
12. AWS references	16

Verified against AWS documentation: September 9, 2026
Audience: Application developers and AWS account administrators
Result: Capture Mantle inference API calls as CloudTrail data events and view them in CloudWatch Logs.

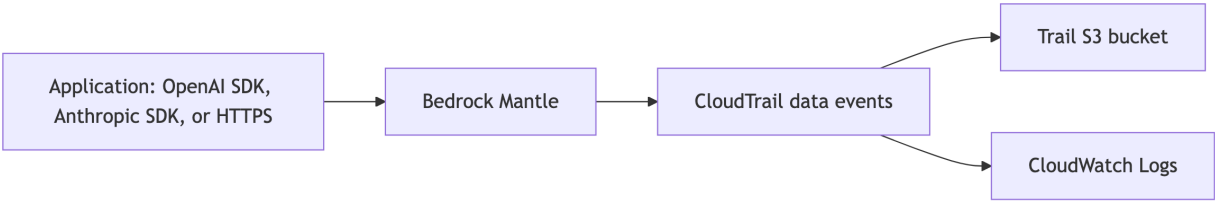
1. Choose the logging mechanism

For requests sent to `bedrock-mantle.{region}.api.aws`, configure:

CloudTrail trail → Mantle data event selectors → CloudWatch Logs delivery.

AWS explicitly states that Bedrock **model invocation logging**, which can capture model inputs and outputs, supports the `bedrock-runtime` endpoint and does **not** currently capture requests to `bedrock-mantle`. Enabling the Bedrock console's model invocation logging setting, or calling `PutModelInvocationLoggingConfiguration`, does not enable this feature for Mantle. [1](#)

Requirement	Mechanism
Audit who called Mantle, when, and which model was requested	CloudTrail data events, delivered to CloudWatch Logs
Monitor aggregate inference volume, tokens, and client errors	CloudWatch metrics in AWS/BedrockMantle
Record complete prompts, responses, or reasoning text from Mantle	Separate application-side logging; CloudTrail is not a payload recorder



What “from the client side” means

An administrator can perform the one-time setup **from a workstation, CloudShell, or automation using the AWS CLI**. This changes AWS account resources.

The inference request itself has **no documented flag that enables CloudTrail capture**. After the trail is configured and logging is started, matching Mantle calls are captured regardless of whether they originate from an SDK, CLI, or direct HTTPS client. Application callers need their normal Mantle inference permissions — `bedrock-mantle:CreateInference` and `bedrock-mantle:CallWithBearerToken`, detailed in section 2 — but they do not need CloudTrail administration or CloudWatch write permissions for this audit path. [2](#)

Client-supplied metadata can help correlate an invocation with an application request. It does not turn logging on.

This guide provides setup commands for you to run. The commands were reviewed and syntax-checked, but were not executed against an AWS account. No AWS resources were created while preparing this guide.

2. Prerequisites and scope

Choose either the **console setup in section 4** or the **CLI setup in section 5**, then run the verification in sections 7–8.

You need:

- 1. An AWS account and a Region where your chosen Mantle model is available.

2. An existing working Mantle inference client or a Bedrock API key authorized to invoke the test model.
3. For CLI setup, AWS CLI v2 and a configured AWS profile with administrative permissions for the resources below. A Bedrock API key alone cannot administer CloudTrail, IAM, S3, or CloudWatch Logs.
4. Python 3 and the `openai` package if you use the optional Python smoke test.

The setup identity needs permissions appropriate to its task:

Task	Relevant permissions
Configure and inspect the trail	<code>cloudtrail:CreateTrail</code> , <code>cloudtrail:UpdateTrail</code> , <code>cloudtrail:PutEventSelectors</code> , <code>cloudtrail:StartLogging</code> , <code>cloudtrail:StopLogging</code> , <code>cloudtrail:GetTrail</code> , <code>cloudtrail:GetTrailStatus</code> , <code>cloudtrail:GetEventSelectors</code>
Create the log destination	<code>logs:CreateLogGroup</code> , <code>logs:PutRetentionPolicy</code>
Create the delivery role	<code>iam:CreateRole</code> , <code>iam:PutRolePolicy</code> , and <code>iam:PassRole</code> for the CloudTrail delivery role
Create and configure the dedicated bucket	<code>s3:CreateBucket</code> , <code>s3:PutBucketOwnershipControls</code> , <code>s3:PutBucketPolicy</code> , <code>s3:PutBucketPublicAccessBlock</code> , <code>s3:PutEncryptionConfiguration</code> , and <code>s3:PutLifecycleConfiguration</code> if you set an S3 lifecycle
Read and query CloudWatch logs	As needed: <code>logs:DescribeLogGroups</code> , <code>logs:DescribeLogStreams</code> , <code>logs:GetLogEvents</code> , <code>logs:FilterLogEvents</code> , <code>logs:StartQuery</code> , <code>logs:GetQueryResults</code>

`cloudtrail:StopLogging` is needed only for the teardown step in section 10. `s3:PutBucketOwnershipControls` is required in addition to `s3:CreateBucket` because the section 5.2 request sets `--object-ownership`, which sends the `x-amz-object-ownership` header. It is **not** included in the `AWSCloudTrail_FullAccess` managed policy, so an identity relying on that policy alone fails on the first command in section 5.2. [16](#)

The **inference caller** is a separate identity from the setup identity above, and needs only: [14](#)

Action	Resource
<code>bedrock-mantle:CreateInference</code>	<code>arn:aws:bedrock-mantle:{region}:{account-id}:project/*</code>
<code>bedrock-mantle:CallWithBearerToken</code>	<code>*</code>

`CallWithBearerToken` is a permission-only action with no matching API operation, and it is required for the Bedrock API key authentication used by the section 7 smoke test. Omitting it is a common cause of an access-denied response on an otherwise correct inference request. The AWS managed policy `AmazonBedrockMantleInferenceAccess` grants both, plus Mantle `Get*/List*`. Add `GetInference`, `CancelInference`, `DeleteInference`, or `CountTokens` only if your application calls those routes.

These are operation-level prerequisites, not a complete IAM policy for every console workflow. Console discovery, organizational controls, permission boundaries, or customer-managed KMS keys may require additional permissions.

CLI example scope: A new, dedicated, single-account, single-Region trail in the commercial AWS partition. Its S3 bucket and CloudWatch log group are in that account and Region. CloudWatch retention is 30 days. It uses S3-managed encryption and captures Mantle `CreateInference` events only.

Console difference: AWS creates multi-Region trails through the console. Their CloudWatch log group is in the trail's home Region and receives selected events from enabled Regions. Use the console-generated delivery role for that scope. The CLI role below deliberately covers only the single-Region example. [3](#)

For an application deployed across Regions, use a multi-Region trail with the appropriate delivery-role scope, or configure coverage in every Region it uses.

3. What Mantle records

Mantle inference calls are **CloudTrail data events**, disabled by default and subject to data-event charges. Their event source is:

```
bedrock-mantle.amazonaws.com
```

AWS documents these inference events: [2](#)

CloudTrail event	Documented route	Selected request parameters
CreateInference	POST /v1/chat/completions	model, stream, service_tier, metadata
CreateInference	POST /v1/responses	model, previousResponse, stream, service_tier, metadata
CreateInference	POST /v1/responses/compact	model, previous_response_id
CreateInference	POST /anthropic/v1/messages	model, stream, max_tokens
CreateInference	POST /v1/embeddings	model
GetInference	GET /v1/responses/{response_id}	response, stream
CancelInference	POST /v1/responses/{response_id}/cancel	response
DeleteInference	DELETE /v1/responses/{response_id}	response
CountTokens	POST /anthropic/v1/messages/count_tokens	model

The table reproduces the route forms in AWS's CloudTrail documentation. Follow your **model card** for the client base URL: for example, Gemma 4 31B uses `https://bedrock-mantle.{region}.api.aws/openai/v1`. Do not change a working model-specific URL merely to match the generic /v1 routes above. [10](#)

Events include identity information, event time, Region, source IP, user agent, resource information, and selected request parameters. AWS adds `callWithBearerToken` and, when applicable, `bearerTokenType`.

CloudTrail does not provide the complete prompt, generated answer, or reasoning transcript. AWS's example has `responseElements: null`. Do not infer thinking mode or token usage from the presence or absence of response text in an audit event.

Metadata is logged verbatim. Use non-sensitive identifiers such as `client_trace_id`, application name, and environment. Do not put prompts, credentials, or personal data into metadata just to make them appear in CloudTrail. [2](#)

4. Option A – Configure with the AWS console

Step 1 – Create or select a trail

1. Open the [CloudTrail console](#).
2. Select the intended home Region.
3. Open **Trails** → **Create trail**.
4. Enter a descriptive name, such as `mantle-inference-audit`.
5. Under **Storage location**, create a dedicated S3 bucket or select an existing bucket with an appropriate CloudTrail bucket policy. A trail still requires S3 storage when CloudWatch delivery is enabled.
6. Choose encryption according to your account requirements. The console defaults to SSE-KMS; if you select it, configure the KMS key permissions. Without SSE-KMS, CloudTrail uses SSE-S3.
7. Keep log file validation enabled.

If you already have a trail, use section 6 before changing its event selectors. [6](#)

Step 2 — Enable CloudWatch Logs delivery

1. In **CloudWatch Logs**, select **Enabled**.
2. Create a log group, for example `/aws/cloudtrail/bedrock-mantle`, or select an existing one.
3. Create a new IAM delivery role using the console, or select a role that:
 - Trusts `cloudtrail.amazonaws.com`.
 - Allows `logs:CreateLogStream` and `logs:PutLogEvents` for the destination streams.
4. Continue to **Choose log events**.

For an existing trail, open the trail, find **CloudWatch Logs** → **Edit**, enable delivery, and save the log group and role. The log group must be in the same account and home Region as the trail. [3](#)

Step 3 — Select Mantle inference data events

1. Enable **Data events**.
2. Use **advanced event selectors**.
3. Add a selector for `AWS::BedrockMantle::Project`.
4. Choose the **Custom** template and configure:

Field	Operator	Value
<code>eventCategory</code>	Equals	Data
<code>resources.type</code>	Equals	<code>AWS::BedrockMantle::Project</code>
<code>eventName</code>	Equals	<code>CreateInference</code>

The console may populate the category and resource type through the event-type selection.

5. Add separate data-event selectors with the same category and event-name filter for:

- `AWS::BedrockMantle::CustomizedModel`
- `AWS::BedrockMantle::Reservation`

These three types are the ones AWS uses in its Mantle data-event example. AWS documents three further Mantle resource types — see the coverage note in section 5.5 before treating this as complete audit coverage.

6. For a dedicated inference-only trail, leave management events unselected. This does not disable your account's free management-event history or change other trails.
7. Review the configuration and create the trail. Confirm that its status is **Logging**.
8. Open the CloudWatch log group and set the desired retention, such as **30 days**.

If the console does not expose the Mantle resource types, create the trail and then use the CLI selectors in section 5.5, following section 6 if that trail already has selectors you need to retain.

The `CreateInference` filter includes the inference routes in section 3, not just Chat Completions. To include the inference lifecycle, add `GetInference`, `CancelInference`, `DeleteInference`, and `CountTokens` to the event-name values. Removing the event-name filter also captures matching file data events. [2](#)

5. Option B — Configure from a workstation with AWS CLI

Run the following steps in order in **one Bash session**. Use new resource names and stop if a command fails. These are setup commands, not commands to execute before every inference.

5.1 — Set the account, Region, and resource names

Replace your-configured-profile with your AWS CLI profile. If it uses IAM Identity Center, complete `aws sso login --profile your-configured-profile` first.

```
MANTLE_AWS_PROFILE="your-configured-profile"
export MANTLE_REGION="us-east-1"

aws_mantle_audit() {
  aws --profile "$MANTLE_AWS_PROFILE" \
    --region "$MANTLE_REGION" \
    --no-cli-pager "$@"
}

aws_mantle_audit sts get-caller-identity

MANTLE_ACCOUNT_ID="$(aws_mantle_audit sts get-caller-identity \
  --query Account --output text)"
MANTLE_TRAIL_NAME="mantle-inference-audit"
MANTLE_LOG_GROUP="/aws/cloudtrail/bedrock-mantle"
MANTLE_ROLE_NAME="CloudTrailMantleLogs-${MANTLE_REGION}"
MANTLE_BUCKET_NAME="mantle-audit-${MANTLE_ACCOUNT_ID}-${MANTLE_REGION}"
MANTLE_TRAIL_ARN="arn:aws:cloudtrail:${MANTLE_REGION}:${MANTLE_ACCOUNT_ID}:trail/${MANTLE_TRAIL_NAME}"
MANTLE_ROLE_ARN="arn:aws:iam:${MANTLE_ACCOUNT_ID}:role/${MANTLE_ROLE_NAME}"
MANTLE_CWL_ARN="arn:aws:logs:${MANTLE_REGION}:${MANTLE_ACCOUNT_ID}:log-group:${MANTLE_LOG_GROUP}:*"

```

Check the account ID before continuing. S3 bucket names are globally unique; change `MANTLE_BUCKET_NAME` if that name already exists. Keep the generated policy files in a new working directory.

5.2 — Create the dedicated S3 bucket

The `us-east-1` bucket-creation request omits `LocationConstraint`; other Regions require it.

```
if [ "$MANTLE_REGION" = "us-east-1" ]; then
  aws_mantle_audit s3api create-bucket \
    --bucket "$MANTLE_BUCKET_NAME" \
    --object-ownership BucketOwnerEnforced
else
  aws_mantle_audit s3api create-bucket \
    --bucket "$MANTLE_BUCKET_NAME" \
    --object-ownership BucketOwnerEnforced \
    --create-bucket-configuration "LocationConstraint=${MANTLE_REGION}"
fi

aws_mantle_audit s3api put-public-access-block \
  --bucket "$MANTLE_BUCKET_NAME" \
  --public-access-block-configuration \
  'BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true'

aws_mantle_audit s3api put-bucket-encryption \
  --bucket "$MANTLE_BUCKET_NAME" \
  --server-side-encryption-configuration \
  '{"Rules": [{"ApplyServerSideEncryptionByDefault": {"SSEAlgorithm": "AES256"}}]}'

```

Create the bucket policy. It authorizes CloudTrail to check the bucket ACL and write this account's trail files, restricted to this trail ARN. [6](#)

```
cat > mantle-cloudtrail-bucket-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {

```

```

    "Sid": "CloudTrailAclCheck",
    "Effect": "Allow",
    "Principal": {"Service": "cloudtrail.amazonaws.com"},
    "Action": "s3:GetBucketAcl",
    "Resource": "arn:aws:s3:::${MANTLE_BUCKET_NAME}",
    "Condition": {
      "StringEquals": {"aws:SourceArn": "${MANTLE_TRAIL_ARN}"}
    }
  },
  {
    "Sid": "CloudTrailWrite",
    "Effect": "Allow",
    "Principal": {"Service": "cloudtrail.amazonaws.com"},
    "Action": "s3:PutObject",
    "Resource": "arn:aws:s3:::${MANTLE_BUCKET_NAME}/AWSLogs/${MANTLE_ACCOUNT_ID}/*",
    "Condition": {
      "StringEquals": {
        "s3:x-amz-acl": "bucket-owner-full-control",
        "aws:SourceArn": "${MANTLE_TRAIL_ARN}"
      }
    }
  }
}
}
EOF

aws_mantle_audit s3api put-bucket-policy \
--bucket "${MANTLE_BUCKET_NAME}" \
--policy file://mantle-cloudtrail-bucket-policy.json

```

This policy assumes a new bucket and no S3 key prefix. For a shared bucket, merge the required statements into its existing policy; do not replace the policy with this example.

5.3 — Create the CloudWatch log group

```

aws_mantle_audit logs create-log-group \
--log-group-name "${MANTLE_LOG_GROUP}"

aws_mantle_audit logs put-retention-policy \
--log-group-name "${MANTLE_LOG_GROUP}" \
--retention-in-days 30

```

CloudWatch retention does not control S3 retention. Set the S3 lifecycle separately according to your audit requirements.

5.4 — Create the CloudTrail delivery role

The trusted service is **CloudTrail**, not Bedrock. [3](#)

```

cat > mantle-cloudtrail-trust-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"Service": "cloudtrail.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }
  ]
}
EOF

```

```
aws_mantle_audit iam create-role \
--role-name "$MANTLE_ROLE_NAME" \
--assume-role-policy-document file://mantle-cloudtrail-trust-policy.json

cat > mantle-cloudtrail-logs-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:${MANTLE_REGION}:${MANTLE_ACCOUNT_ID}:log-group:${MANTLE_LOG_GROUP}:log-stream:${MANTLE_ACCOUNT_ID}_Cl
    ]
  ]
}
EOF

aws_mantle_audit iam put-role-policy \
--role-name "$MANTLE_ROLE_NAME" \
--policy-name CloudTrailMantleLogDelivery \
--policy-document file://mantle-cloudtrail-logs-policy.json
```

This stream pattern follows AWS's single-account, single-Region example. A multi-Region or organization trail requires the corresponding additional stream permissions. [3](#)

5.5 — Create the advanced event selector file

Save the following file. Separate selectors cover the three resource types used by AWS's Mantle data-event example. The event-name filter limits capture to invocation creation.

```
cat > mantle-data-selectors.json <<'EOF'
[
  {
    "Name": "Mantle project inference",
    "FieldSelectors": [
      {"Field": "eventCategory", "Equals": ["Data"]},
      {"Field": "resources.type", "Equals": ["AWS:BedrockMantle::Project"]},
      {"Field": "eventName", "Equals": ["CreateInference"]}
    ]
  },
  {
    "Name": "Mantle customized-model inference",
    "FieldSelectors": [
      {"Field": "eventCategory", "Equals": ["Data"]},
      {"Field": "resources.type", "Equals": ["AWS:BedrockMantle::CustomizedModel"]},
      {"Field": "eventName", "Equals": ["CreateInference"]}
    ]
  },
  {
    "Name": "Mantle reservation inference",
    "FieldSelectors": [
      {"Field": "eventCategory", "Equals": ["Data"]},
      {"Field": "resources.type", "Equals": ["AWS:BedrockMantle::Reservation"]},
      {"Field": "eventName", "Equals": ["CreateInference"]}
    ]
  }
]
EOF
```

Why three selectors? The Mantle page groups all three resource types into a single selector in its example, while the `AdvancedFieldSelector` reference states that a selector can contain only one `resources.type` field and directs you to add another selector for each additional resource type. Separate selectors express the intended coverage without depending on which of those two forms the API accepts. [2](#)

Coverage caveat — three of six resource types. AWS's Mantle CloudTrail page documents six resource types that Mantle events can reference: Project, Reservation, and CustomizedModel, plus `AWS::BedrockMantle::Environment`, `AWS::BedrockMantle::Runtime`, and `AWS::BedrockMantle::Skill`. The selectors above cover only the first three, because those are the ones in AWS's data-event example. Two things are currently unverifiable from the documentation: whether `CreateInference` events ever reference the other three, and whether those three are accepted values for `resources.type`, since the general CloudTrail "Data events supported by CloudTrail" table lists no `BedrockMantle` rows at all.

If your audit requirement is completeness rather than a working example, add a fourth, fifth, and sixth selector for the remaining types and confirm with `get-event-selectors` that CloudTrail accepted them. A rejected value fails the whole `put-event-selectors` call, so test this on the dedicated trail before applying it to a shared one.

2

5.6 — Create the trail, apply selectors, and start logging

```
aws_mantle_audit cloudtrail create-trail \
  --name "$MANTLE_TRAIL_NAME" \
  --s3-bucket-name "$MANTLE_BUCKET_NAME" \
  --no-is-multi-region-trail \
  --no-include-global-service-events \
  --enable-log-file-validation \
  --cloud-watch-logs-log-group-arn "$MANTLE_CWL_ARN" \
  --cloud-watch-logs-role-arn "$MANTLE_ROLE_ARN"

aws_mantle_audit cloudtrail put-event-selectors \
  --trail-name "$MANTLE_TRAIL_NAME" \
  --advanced-event-selectors file://mantle-data-selectors.json

aws_mantle_audit cloudtrail start-logging \
  --name "$MANTLE_TRAIL_NAME"
```

Creating a trail through the CLI is not a substitute for starting logging. Run `start-logging` explicitly. If IAM propagation causes a delivery-role error immediately after role creation, verify the role and policy, allow propagation, and retry the failed command. 13

put-event-selectors replaces the trail's selector configuration. This example intentionally makes the new trail data-only. Do not run that command against an existing shared trail without following section 6. 5

5.7 — Verify the configuration

```
aws_mantle_audit cloudtrail get-trail \
  --name "$MANTLE_TRAIL_NAME" \
  --query 'Trail.{Trail:Name,HomeRegion:HomeRegion,MultiRegion:IsMultiRegionTrail,LogGroup:CloudWatchLogsLogGroupArn,DeliveryRole:CloudWatchLogsDeliveryRole}'

aws_mantle_audit cloudtrail get-event-selectors \
  --trail-name "$MANTLE_TRAIL_NAME"

aws_mantle_audit cloudtrail get-trail-status \
  --name "$MANTLE_TRAIL_NAME" \
  --query '{IsLogging:IsLogging,S3Error:LatestDeliveryError,CloudWatchError:LatestCloudWatchLogsDeliveryError,LastCloudWatchDelivery:LastCloudWatchDeliveryTimestamp}'
```

Check that:

- `IsLogging` is true.
- The destination ARN and role ARN identify the intended resources.
- The advanced selectors contain Data, the Mantle resource types, and `CreateInference`.
- Delivery errors are absent. Delivery timestamps may be empty until an event has been delivered.

6. If you already have a trail

You do not need a second trail if an existing trail can capture these events and deliver them to the required log group.

1. Work in the trail's **home Region**.
2. Save the current trail and selector configuration:

```
aws_mantle_audit cloudtrail get-trail \
  --name "$MANTLE_TRAIL_NAME" \
  --output json > mantle-trail-before.json

aws_mantle_audit cloudtrail get-event-selectors \
  --trail-name "$MANTLE_TRAIL_NAME" \
  --output json > mantle-selectors-before.json
```

3. Inspect `AdvancedEventSelectors` and `EventSelectors` in the saved result:

- If advanced selectors are already configured, prepare a **complete replacement array** containing the selectors you want to retain plus the Mantle selectors. Preserve management-event selection and all existing data-event coverage.
- If basic `EventSelectors` are configured, do not apply an advanced-only file directly. Basic and advanced selectors cannot coexist on the same trail; migrate the existing coverage to advanced selectors first, or use a dedicated trail.
- Advanced selectors support up to **500 total condition values** across all selectors on a trail. This example uses three selectors and nine values: each selector specifies the category, one resource type, and one event name. The five-selector limit described for basic `EventSelectors` should not be presented as an advanced-selector limit. Review the full replacement configuration before applying it. [4](#)

4. Save the reviewed replacement array as `reviewed-complete-selectors.json`, then apply it:

```
aws_mantle_audit cloudtrail put-event-selectors \
  --trail-name "$MANTLE_TRAIL_NAME" \
  --advanced-event-selectors file://reviewed-complete-selectors.json
```

5. If CloudWatch delivery is not configured, create the log group and an appropriately scoped delivery role, then associate them:

```
aws_mantle_audit cloudtrail update-trail \
  --name "$MANTLE_TRAIL_NAME" \
  --cloud-watch-logs-log-group-arn "$MANTLE_CWL_ARN" \
  --cloud-watch-logs-role-arn "$MANTLE_ROLE_ARN"

aws_mantle_audit cloudtrail start-logging \
  --name "$MANTLE_TRAIL_NAME"
```

If delivery is already configured, preserve its destination unless you intend to move it. A trail has one CloudWatch Logs destination; updating it is not adding a second destination.

6. Repeat section 5.7 and confirm that the previously required selectors remain present.

A CloudTrail Lake event data store is a separate capture/query option. Creating one alone does not configure the trail-to-CloudWatch delivery described here. [2](#)

7. Send an invocation from your client

After logging is enabled, make a **new** request. CloudTrail data-event capture does not retroactively recover calls made before capture was enabled.

Existing application

Keep your working Mantle model ID and base URL. For Chat Completions or Responses, add an optional non-sensitive metadata identifier:

```
{
  "metadata": {
    "application": "mantle-audit-smoke-test",
    "client_trace_id": "a-unique-id-for-this-request"
  }
}
```

A successful request without this metadata is still captured when it matches the trail selectors. Metadata support and recorded fields differ by API; do not copy this field blindly into an Anthropic Messages request. [2](#)

Python smoke test using Gemma 4 31B

The following uses the model and endpoint from AWS's Gemma 4 31B model card. Use it only if that model is available to your account in the selected Region. Otherwise, keep your existing working model and its documented base URL. Gemma 4 is Mantle-only. [10](#)

Install the client:

```
python3 -m pip install openai
```

Make your Bedrock API key available as the environment variable MANTLE_BEDROCK_API_KEY through your normal credential mechanism. Do not use an OpenAI API key.

Save this as mantle_logging_smoke_test.py:

```
import json
import os
import time
import uuid

from openai import OpenAI

region = os.environ.get("MANTLE_REGION", "us-east-1")
trace_id = str(uuid.uuid4())

client = OpenAI(
    api_key=os.environ["MANTLE_BEDROCK_API_KEY"],
    base_url=f"https://bedrock-mantle.{region}.api.aws/openai/v1",
    max_retries=0,
)

started = time.perf_counter()
raw_response = client.chat.completions.with_raw_response.create(
    model="google.gemma-4-31b",
    messages=[
        {"role": "user", "content": "Reply with exactly: Mantle logging test."}
    ],
    metadata={
        "application": "mantle-audit-smoke-test",
        "client_trace_id": trace_id,
    },
    stream=False,
)
completion = raw_response.parse()

print(json.dumps({
    "client_trace_id": trace_id,
```

```
"completion_id": completion.id,
"elapsed_ms": round((time.perf_counter() - started) * 1000),
"request_id_headers": {
    key: value
    for key, value in raw_response.headers.items()
    if key.lower() in {
        "x-request-id", "x-amzn-requestid", "x-amzn-request-id"
    }
},
}))
print(completion.choices[0].message.content)
```

Run it:

```
python3 mantle_logging_smoke_test.py
```

The test disables SDK retries so one test run does not silently produce multiple attempts. In a production application, retries may generate multiple audit events for one logical application request.

Record the printed `client_trace_id`. It is the primary correlation key for the query below. The completion ID is not necessarily the CloudTrail requestID; use recorded metadata instead of assuming that the identifiers match.

No reasoning setting is required for this logging test. The test does not validate Gemma's thinking-mode mapping.

8. Find the event in CloudWatch Logs

CloudTrail usually delivers events to CloudWatch Logs in **about five minutes on average**, but AWS does not guarantee that delay. Use a query time range that includes the test and allow for delivery time. [3](#)

Console and Logs Insights

1. Open the [CloudWatch console](#) in the **log group's Region**.
2. Select **Logs** → **Log groups** and open the configured group.
3. Inspect a CloudTrail stream, or open **Logs Insights**, select the log group, and choose **Logs Insights QL**.
4. Set the time range to the last 30–60 minutes.
5. Run:

```
fields @timestamp, eventTime, eventName, awsRegion,
      userIdentity.arn, requestParameters.model,
      requestParameters.metadata.client_trace_id, requestID
| filter eventSource = "bedrock-mantle.amazonaws.com"
| filter eventCategory = "Data" and eventName = "CreateInference"
| sort @timestamp desc
| limit 100
```

To find the smoke test, add this filter before sort, replacing the placeholder:

```
| filter requestParameters.metadata.client_trace_id = "PASTE_PRINTED_TRACE_ID"
```

To count audited calls by model:

```
filter eventSource = "bedrock-mantle.amazonaws.com"
and eventCategory = "Data"
and eventName = "CreateInference"
| stats count(*) as auditEvents by requestParameters.model
| sort auditEvents desc
```

This counts audit events. It is not a measurement of successful completions, billable tokens, or unique end-user requests.

CLI

From the Bash session configured in section 5:

```
MANTLE_START_MS="$(python3 -c 'import time; print(int((time.time() - 3600) * 1000))')"

aws_mantle_audit logs filter-log-events \
  --log-group-name "$MANTLE_LOG_GROUP" \
  --start-time "$MANTLE_START_MS" \
  --filter-pattern '{ $.eventSource = "bedrock-mantle.amazonaws.com" && $.eventName = "CreateInference" }' \
  --query 'events[].message' \
  --output json
```

Do not use CloudTrail Event history or `aws cloudtrail lookup-events` to verify Mantle inference data events. Event history covers management events, not data events. Use CloudWatch Logs, the trail's S3 files, or an appropriately configured CloudTrail Lake event data store. [9](#)

Expected event shape

This is an illustrative subset, not a captured test result:

```
{
  "eventSource": "bedrock-mantle.amazonaws.com",
  "eventName": "CreateInference",
  "eventCategory": "Data",
  "awsRegion": "us-east-1",
  "requestParameters": {
    "model": "google.gemma-4-31b",
    "stream": false,
    "metadata": {
      "application": "mantle-audit-smoke-test",
      "client_trace_id": "your-generated-uuid"
    }
  },
  "callWithBearerToken": true,
  "bearerTokenType": "ABSK"
},
"responseElements": null,
"requestID": "service-generated-request-id",
"resources": [
  {
    "type": "AWS::BedrockMantle::Project",
    "ARN": "arn:aws:bedrock-mantle:us-east-1:111122223333:project/example-project"
  }
]
}
```

Identity fields, resource ARNs, bearer-token details, and optional parameters depend on the actual request. Inspect your event rather than expecting every example field.

Completion criteria

The setup is working when a newly issued request produces a CloudWatch event with:

- `eventSource = bedrock-mantle.amazonaws.com`
- `eventCategory = Data`
- `eventName = CreateInference`

- The expected Region and model
- The test's `requestParameters.metadata.client_trace_id`, if supplied

9. Troubleshooting

Symptom	Check and action
Model invocation logging is enabled, but Mantle prompts/responses are absent	That feature does not capture Mantle. Configure CloudTrail data events for auditing, or application logging for payloads.
ListModels appears but inference calls do not	ListModels is a management event. Enable the Mantle data selectors and generate a new invocation.
Event history shows no CreateInference	Expected: Event history does not show data events. Query the configured CloudWatch log group.
No log streams or events	Confirm <code>IsLogging=true</code> , the selectors, the destination ARN, a new matching request, and sufficient delivery time.
LatestCloudWatchLogsDeliveryError is populated	Check that CloudTrail can assume the role, that it has <code>CreateLogStream</code> and <code>PutLogEvents</code> , and that stream ARNs cover the account/Region scope.
LatestDeliveryError is populated	Check the S3 bucket policy, trail ARN condition, bucket/key prefix, and any KMS permissions.
CloudTrail configuration fails with access denied	The setup identity needs AWS credentials and configuration permissions, including <code>iam:PassRole</code> . A Bedrock API key does not provide those permissions.
<code>create-bucket</code> fails with access denied at the start of section 5.2	<code>--object-ownership</code> requires <code>s3:PutBucketOwnershipControls</code> in addition to <code>s3:CreateBucket</code> . It is absent from <code>AWSCloudTrail_FullAccess</code> . 16
The inference request itself returns access denied	This is a caller-permission failure, not a logging failure. The calling identity needs <code>bedrock-mantle:CreateInference</code> on the project ARN and <code>bedrock-mantle:CallWithBearerToken</code> . See section 2. 14
Invocation is successful but filtered out	Verify the actual endpoint, account, Region, resource types, and any <code>resources.ARN</code> or event-name restrictions. Start without optional ARN filters. If the event's <code>resources[].type</code> is a Mantle type not in your selectors, see the coverage caveat in section 5.5.
Events seem to be in the wrong Region	A single-Region trail captures its Region; a multi-Region trail delivers to the log group in its home Region. The event's <code>awsRegion</code> identifies where the call occurred.
Existing audit coverage disappears after setup	<code>put-event-selectors</code> replaced the configuration. Rebuild a complete selector array using the saved configuration and intended Mantle selectors.
Several events correspond to one application action	Check client retries, multiple inference calls, and duplicate trail delivery. Use the application correlation ID and CloudTrail eventID appropriately.
Prompt, response, or reasoning text is missing	Expected for this audit mechanism. CloudTrail's selected request fields are not full model payloads.

Use the `status` command in section 5.7 to inspect delivery errors. AWS documents `LatestDeliveryError`, `LatestCloudWatchLogsDeliveryError`, and `LatestCloudWatchLogsDeliveryTime` specifically for this purpose. [12](#)

10. Operational metrics, retention, and cost

CloudWatch metrics are separate

Mantle publishes service metrics under **AWS/BedrockMantle** independently of this CloudTrail setup. In CloudWatch, select **Metrics** → **All metrics** → **AWS/BedrockMantle** after traffic has been generated. [8](#)

Documented metrics include:

- Inferences and InferenceClientErrors, at the Account, Project, Model, and Project+Model levels

- TotalInputTokens and TotalOutputTokens, at the Account, Project, and Model levels — not Project+Model
- InputTokens and OutputTokens, at the Project+Model level only, one datum per inference, for percentile statistics

Use the documented dimension sets: account level has no extra dimensions; the other levels use Project, Model, or both. Avoid summing different dimension levels together, because a request can contribute to multiple levels.

Mantle is in-Region only. Metrics are emitted in the Region that handled the request and are not aggregated across Regions the way cross-Region inference traffic is on bedrock-runtime. [8](#)

Correction to the supplied reference answer: AWS currently states that equivalents of InvocationLatency and TimeToFirstToken are **not yet published** for Mantle. Measure latency in the application if required. The smoke test measures end-to-end elapsed time; it does not measure time to first token. [8](#)

Charges and retention

- CloudTrail **data events incur charges**. Additional trails capturing the same traffic can create additional event-delivery costs.
- CloudWatch Logs ingestion, retention/storage, and Logs Insights queries have their own charges.
- The trail's S3 storage and requests also have charges; customer-managed KMS encryption may add KMS charges.
- Management events in the 90-day CloudTrail Event history are available at no additional charge. This is not a blanket statement that every management-event copy or delivery destination is free. [2](#)
- The 30-day CloudWatch retention in this example is independent of S3 lifecycle retention.
- Capture begins after configuration is active; enabling capture does not backfill earlier inference calls.

To stop future collection for the dedicated test trail while retaining existing log objects:

```
aws_mantle_audit cloudtrail stop-logging \  
--name "$MANTLE_TRAIL_NAME"
```

Stopping collection does not remove the trail, bucket, log group, IAM role, or stored data. Existing storage can continue to incur charges. Do not stop a shared trail simply to disable this one application's auditing.

11. If you require client-side prompt and response logs

For complete Mantle payloads, add a separate application logging path:

1. Provision a distinct log group, such as /applications/my-app/mantle-invocations, with an explicit retention period.
2. Instrument the SDK call to record the request correlation ID, model, start/end time, result or error, and the input/output fields your application actually needs.
3. Redact or omit credentials and sensitive content before logging.
4. Deliver those records through your application's existing CloudWatch logging integration or through the CloudWatch Logs API. Direct API delivery needs separate application AWS credentials with logs:CreateLogStream and logs:PutLogEvents on that log group, plus logs:CreateLogGroup and logs:PutRetentionPolicy if the application provisions the group itself.
5. For streaming calls, explicitly collect the desired output chunks and distinguish partial output from a completed response.

These records are **application-generated logs**, separate from CloudTrail audit events. A Bedrock API key does not itself grant permission to write arbitrary CloudWatch logs. Enabling CloudTrail does not implement this application logging path.

12. AWS references

1. [Monitor model invocation using CloudWatch Logs and Amazon S3](#) — Runtime-only model payload logging; Mantle excluded.
2. [Monitor bedrock-mantle API calls using CloudTrail](#) — Event names, metadata, resource types, data-event capture, and sample event.
3. [Sending CloudTrail events to CloudWatch Logs](#) — Console/CLI integration, delivery role, account/Region scope, and delivery timing.
4. [CloudTrail AdvancedFieldSelector API reference](#) — Selector fields and resource-type constraints.
5. [AWS CLI: cloudtrail put-event-selectors](#) — Replacement behavior, basic/advanced selector restrictions, and limits.
6. [Amazon S3 bucket policy for CloudTrail](#) — CloudTrail bucket access and trail ARN conditions.
7. [Creating a trail with the CloudTrail console](#) — Multi-Region console workflow, data events, encryption, and destinations.
8. [Monitor bedrock-mantle inference using CloudWatch metrics](#) — Namespace, metric names, dimensions, and current latency-metric limitation.
9. [Working with CloudTrail event history](#) — 90-day management-event history and data-event exclusion.
10. [Gemma 4 31B model card](#) — Mantle availability and model-specific /openai/v1 base URL.
11. [AWS CLI: cloudtrail create-trail](#) — Trail creation parameters.
12. [AWS CLI: cloudtrail get-trail-status](#) — Logging state and delivery diagnostics.
13. [Creating a trail with the AWS CLI](#) — Explicitly starting logging after CLI trail creation.
14. [Actions, resources, and condition keys for Amazon Bedrock Powered by AWS Mantle](#) — bedrock-mantle IAM actions, the CallWithBearerToken permission-only action, and resource ARN formats.
15. [AmazonBedrockMantleInferenceAccess](#) — AWS managed policy granting Mantle inference permissions.
16. [Amazon S3 CreateBucket API reference](#) — Permissions required when CreateBucket includes the x-amz-object-ownership header.